

SEMANTiCS Scope Explorer: Query-Conditioned KG-RAG Summaries of Czech Public-Health Tables

SEMANTiCS Project Team¹

¹SEMANTiCS project workspace

Abstract

SEMANTiCS Scope Explorer is a reproducible demo for turning Czech natural language questions over large public-health tables into inspectable aggregate SQL summaries. To our knowledge, it is the first RAG/KG-RAG analysis demo built for this exact combination of Czech NZIS infectious-disease aggregates and MKN-10-CZ code-list data. The system compares a flat RAG baseline with KG-RAG over the same 272,858-row DuckDB fact table, so the difference is scope construction rather than database access. Flat RAG selects table-present values from retrieved text. KG-RAG adds typed MKN-10-CZ hierarchy expansion before SQL, converting disease-family phrases into complete diagnosis-code scopes. On ten saved live demo queries, KG-RAG improves diagnosis micro-F1 from 0.545 to 0.935, macro-F1 from 0.567 to 0.921, exact diagnosis scopes from 1/10 to 8/10, and complete-filter accuracy from 1/10 to 6/10, while producing no false-positive diagnosis codes. The demo exposes the retrieved evidence, graph expansion, generated SQL, and returned table, making the semantic gain and remaining failure modes directly auditable.

Keywords

retrieval-augmented generation, knowledge graphs, KG-RAG, semantic retrieval, text-to-SQL, public-health open data, tabular summarization

1. Problem and Contribution

Public-health table exploration is often framed as document search, but the hard part is usually scope construction. Even a simple user phrase such as “děti” (children) is not a database predicate. The source table stores diagnosis codes, official age-group and region labels, months, years, and one reported aggregate measure. A fluent answer is not useful if it silently omits sibling diagnoses or applies informal values that cannot be executed.

SEMANTiCS Scope Explorer treats question answering as query-conditioned summarization over a table, not as prose generation over retrieved chunks. The demo contributes a shared DuckDB answer layer, a flat RAG baseline, a KG-RAG variant with typed MKN-10-CZ hierarchy expansion, and a saved ten-query evaluation. To our knowledge, no prior demo has reported RAG-versus-KG-RAG scope accuracy on this exact NZIS infectious-disease source joined to the Czech MKN-10-CZ terminology. The comparison is controlled: both methods query the same `fact_infectious_disease_cases_enriched` table, and both return selected entities, generated SQL, and a rendered result table. The reported measure is always `reported_case_count` (source `pocet_pripadu`); it is not interpreted as unique patients, incidence, prevalence, causal evidence, or clinical guidance.

2. Architecture

The current SQL answer layer intentionally focuses on a newly built infectious-disease fact table rather than claiming to answer over every profiled source. The final answer table contains 272,858 infectious-disease rows from 2018–2025, enriched with MKN-10-CZ terminology, derived MKN blocks, chapter labels, age intervals, region labels, sex labels, and a stable measure. MKN-10-CZ (39,136 rows) and the NZIS age-group code list (228 rows) are joined into this table. The 1,418,107-row hospitalization dataset is loaded into DuckDB and represented in the KG through diagnosis, year, and measure context, but it is not yet joined into the final SQL answer table. This keeps the RAG/KG-RAG comparison controlled on

the new infectious-disease dataset build; cross-fact analyses with hospitalization are the next step. All sources come from Czech public-health open data [1]. DuckDB provides the local analytical execution layer [2].

The AI Scope Builder has four shared runtime stages. First, it extracts candidate spans from the user’s Czech question. Second, it embeds each span and compares it with a flat catalog of public values and grouping columns. The database candidate score is

$$\text{score}_{DB} = 0.86 \cos(e_s, e_c) + 0.14 \text{lex}(s, c),$$

with candidates below cosine 0.20 discarded. Third, a model selects filters, groupings, and context entities from candidate IDs. Deterministic safeguards promote at most five diagnosis filters per disease/code span, require score at least 0.28, and allow no more than a 0.14 drop from the best diagnosis candidate. Fourth, SQL generation is constrained to a single read-only SELECT over the shared final table. The AI methods are not allowed to filter directly on MKN block or chapter columns; hierarchy must be converted into explicit `diagnosis_code IN (...)` values.

Flat RAG uses 114 embedded documents, one per table-present infectious diagnosis. KG-RAG uses 40,906 node documents and a graph with 40,906 nodes and 87,364 edges. The graph contains MKN chapters, derived MKN blocks, MKN codes, infectious-disease diagnosis values, hospitalization diagnosis values, years, months, measures, datasets, and columns. Its main expansion edges are `CONTAINS_MKN_CODE`, `HAS_TABLE_DIAGNOSIS_VALUE`, and `SAME_AS_MKN_CODE`; it also stores 2,418 weighted co-occurrence edges and 1,169 diagnosis profile-similarity edges.

KG-RAG inserts one extra step between flat entity selection and SQL. Selected diagnosis seeds are grouped by MKN block and ranked by

$$\text{score}_{block} = 0.55 \frac{|C_b|}{|C|} + 0.25 \frac{\sum_{d \in C_b} w_d}{\sum_{d \in C} w_d} + 0.20 \text{lex}(q, b).$$

Here C is the selected seed set, C_b its subset inside block b , and w_d is the reported-case total for diagnosis d . If block lexical similarity is at least 0.65, or no seed code is already present in the table for that block, the whole table-present block is used; otherwise the system uses a contiguous seed range or seed-only expansion. These ratios are deterministic demo parameters chosen through interactive user-experience tuning with the live demo. They are not learned model parameters and were not fit by optimizing the evaluation metrics.

3. Evaluation

The saved evaluation in `dg_evaluation` replays ten Czech user queries against live API outputs for RAG and KG-RAG. Gold diagnosis groups such as A00–A09, B00–B09, B15–B19, and J12–J18 are resolved by intersecting the official range with diagnosis codes present in the DuckDB fact table. Other gold filters are official year, age-group, and region values from the evaluation JSON and DuckDB lookups. The scorer parses SQL WHERE clauses first; if an expected filter is not explicit in SQL but appears in the returned rows, the distinct returned row values are used. Complete-filter accuracy is binary per question: every expected filter column must have exactly the expected values and no unexpected filter column.

Table 1 shows the central result. KG-RAG doubles micro recall from 0.439 to 0.878 and raises exact diagnosis scopes from one query to eight. More importantly for a public demo, KG-RAG produces zero false-positive diagnosis codes across the ten queries, while flat RAG adds 14 wrong diagnosis codes. The main win is not a nicer answer sentence; it is the correct set of values over which a huge table is summarized.

Table 2 gives the stricter table-summarization view: all filters must be right, not only disease family. KG-RAG still improves accuracy from 0.10 to 0.60. The remaining failures are informative. EQ-03 and EQ-05 are not diagnosis-hierarchy failures; they expose that age and regional aliases are currently

Table 1

Diagnosis-scope effectiveness. Values are precision/recall/F1; Ex is exact diagnosis-set match.

ID	Target	RAG	Ex	KG-RAG	Ex
EQ-01	A00–A09	.60/.33/.43	0	1.00/1.00/1.00	1
EQ-02	A00–A09	.60/.33/.43	0	1.00/1.00/1.00	1
EQ-03	A00–A09	.60/.33/.43	0	1.00/1.00/1.00	1
EQ-04	A00–A09	.60/.33/.43	0	1.00/1.00/1.00	1
EQ-05	A00–A09	.60/.33/.43	0	1.00/1.00/1.00	1
EQ-06	A00–A09	.60/.33/.43	0	1.00/.56/.71	0
EQ-07	B00–B09	.60/.33/.43	0	1.00/.33/.50	0
EQ-08	B15–B19	1.00/1.00/1.00	1	1.00/1.00/1.00	1
EQ-09	J12–J18	1.00/.71/.83	0	1.00/1.00/1.00	1
EQ-10	J12–J18	1.00/.71/.83	0	1.00/1.00/1.00	1
All	macro	.72/.48/.57	1/10	1.00/.89/.92	8/10
All	micro	.72/.44/.55	–	1.00/.88/.94	–

Table 2

Complete-filter effectiveness. A correct run must match all expected diagnosis, time, age, and region filters.

ID	Expected filters	RAG	KG-RAG	KG-RAG residual when incorrect
EQ-01	diagnosis	0	1	–
EQ-02	diagnosis + year	0	1	–
EQ-03	diagnosis + age filter	0	0	age filter broadened to all groups
EQ-04	diagnosis + age filter	0	1	–
EQ-05	diagnosis + region filter	0	0	region filter broadened to all regions
EQ-06	diagnosis + region filter	0	0	region kept; diagnosis scope partial
EQ-07	diagnosis	0	0	B00–B09 scope partial
EQ-08	diagnosis	1	1	–
EQ-09	diagnosis	0	1	–
EQ-10	diagnosis	0	1	–
All	complete filters	1/10	6/10	–

handled by the shared flat scope builder rather than by a graph of age and region concepts. EQ-06 and EQ-07 are partial hierarchy expansions caused by the selected seeds and conservative block-expansion policy. The demo therefore shows both a strong effectiveness gain and an auditable path for the next engineering iteration.

4. Discussion and Conclusion

The results support the paper’s core claim: KG-RAG is valuable for query-conditioned table summarization when a user phrase denotes a coded family rather than a literal table value. The A00–A09 and J12–J18 cases show why graph closure matters. Many flat RAG approaches retrieve plausible high-precision seeds but fail to retrieve the complete controlled-vocabulary set, so recall becomes the weak point even when the top retrieved labels look correct. Our flat baseline follows this pattern: it often finds relevant diagnoses, but it misses sibling codes needed for complete table summarization and sometimes introduces outsiders. KG-RAG turns the same initial evidence into an explicit code set, then lets DuckDB compute the answer. This separation of semantic scope construction from SQL execution is what makes the demo inspectable and repeatable.

The current limitations are also clear. MKN blocks are configured from known ranges because the source CSV provides chapters but no explicit block rows. The graph does not yet model age concepts or regional aliases; those filters use shared flat candidate selection. The SQL answer layer also does not yet expose all loaded datasets as answer facts: hospitalization is available as source and graph context, while infectious-disease reported cases are the current summarized measure. The numeric ratios in candidate ranking and hierarchy ranking are transparent demo-tuned settings, not trained parameters. Future work should add typed age/region concept nodes, extend the new dataset build into multi-fact hospitalization analyses, learn or calibrate the ratios on held-out questions, and evaluate

larger multilingual query sets. The system uses aggregate public-health data and does not provide clinical advice, causal claims, or patient-level inference.

Declaration on Generative AI

During preparation of this manuscript draft, the authors used an AI coding assistant to transform project artifacts into $\LaTeX$ prose and to check compilation. The authors are responsible for reviewing, editing, and validating the final content before submission.

References

- [1] Institute of Health Information and Statistics of the Czech Republic, Ministry of Health of the Czech Republic, Czech public-health open data: Infectious diseases, age groups, mkn-10-cz, and hospitalization records, 2026. CSV and metadata files cached in the SEMANTiCS workspace on 13 June 2026.
- [2] M. Raasveldt, H. Muhleisen, Duckdb: An embeddable analytical database, in: Proceedings of the 2019 International Conference on Management of Data, 2019, pp. 1981–1984. doi:10.1145/3299869.3320212.